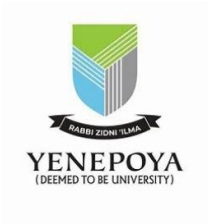
IBM Innovation Centre for Education

NCS Nihon Communication Solutions



Yenepoya Institute of Arts Science Commerce and Management

Yenepoya Deemed to be University

Final Project Report

on

Smart PG Finder – AI-Powered Paying Guest Recommendation System

Student Name: Shaima Sharif Kulavoor **Roll No: 23BSAMD030**

Industry Mentor: Mr. Sumith Kumar Shukla	**Guided by:** Ankhitha Maam

Table of Contents

Executive Summary

Background

The rapid urbanization of Indian metropolitan cities has created an acute demand for affordable, well-managed paying guest (PG) accommodations. Students migrating to education hubs and working professionals relocating for career opportunities face significant challenges in finding suitable housing that aligns with their budgetary constraints, lifestyle preferences, and proximity requirements. Conventional methods of PG discovery rely on word-of-mouth, physical notice boards, or generic classified platforms that lack intelligent personalization.

This project, Smart PG Finder, was conceived to address this gap by deploying a full-stack, AI-powered recommendation platform. The application aggregates over 4,746 real property listings drawn from a comprehensive House Rent Dataset covering six major Indian cities—Bangalore, Chennai, Delhi, Kolkata, Mumbai, and Pune—and presents them through a modern, responsive web interface backed by a machine-learning recommendation engine.

Aim

The primary aim of this project is to design, develop, and deploy an intelligent paying guest recommendation system that enables users to discover accommodations tailored to their individual preferences. The system combines content-based collaborative filtering through scikit-learn's CountVectorizer and cosine similarity algorithms with a natural language chatbot powered by the Groq API (Mixtral-8x7b-32768 model). The platform provides users with end-to-end property discovery, direct owner contact, and a streamlined booking workflow.

Technologies

The project employs a modern, layered technology stack. The backend is built with Flask 2.3.2 (Python REST API) and SQLite for persistent storage via SQLAlchemy ORM. User authentication is handled through JSON Web Tokens (JWT) with 7-day expiry. Machine learning components rely on scikit-learn 1.4.0 for recommendation logic and pandas 2.2.2 for data manipulation. The frontend is a React 18 single-page application styled with Tailwind CSS 3, using Axios for HTTP communication and React Context API for global state management.

Hardware Architecture

The application is designed to run on standard commodity hardware. The backend Flask server operates on a host machine with a minimum of 4 GB RAM and a dual-core processor, serving HTTP requests on port 5000. The SQLite database is stored as a file-based artifact (database.db) within the application directory. The ML recommendation model is held in-process memory during application runtime, requiring no GPU resources. In production, the system is designed for migration to cloud infrastructure such as AWS EC2 or Google Cloud Compute Engine with a PostgreSQL backend.

Software Architecture

The software follows a three-tier Model-View-Controller (MVC) architecture. The data layer comprises SQLite tables managed via SQLAlchemy ORM models. The service layer includes Flask blueprints (auth_routes, recommendation_routes, chat_routes, booking_routes, upload_routes) that handle business logic. The presentation layer is a React SPA communicating exclusively through REST API endpoints. The ML engine is a standalone Python class (PGRecommender) initialized once at server startup and reused across requests, ensuring O(1) recommendation lookups after the initial cosine similarity matrix computation.

1. Introduction

1.1 Project Background and Motivation

India's housing rental market, particularly the paying guest segment, has witnessed explosive growth over the past decade. According to industry estimates, over 50 million students and young professionals seek affordable accommodation in Indian cities every year. The fragmented and largely informal nature of PG listings means that property seekers spend disproportionate time and effort searching for suitable accommodation, often relying on outdated classifieds, unreliable brokers, or inefficient manual searches.

The absence of a centralized, intelligent discovery platform creates information asymmetry between property owners and seekers. Smart PG Finder directly addresses this problem by providing a data-driven solution that leverages machine learning to match users with properties that best align with their stated preferences—city, budget, BHK configuration, furnishing status, and tenant type—while providing a frictionless user experience through modern web technologies.

1.2 Problem Statement

The core problem this project aims to solve can be summarized as follows: Property seekers in India lack access to an intelligent, personalized PG recommendation system that integrates real-time search, machine-learning-based property suggestions, natural language querying, and direct owner connectivity in a single unified platform.

Existing platforms either present raw unfiltered listings without personalization, charge prohibitive intermediary commissions, or lack the technical sophistication to understand nuanced user requirements expressed in natural language. The Smart PG Finder fills this void.

1.3 Objectives

- Design and implement a full-stack web application for PG discovery and recommendation using Flask and React.
- Develop a content-based ML recommendation engine using TF-IDF-style feature extraction and cosine similarity on a dataset of 4,746 real property listings.

- Integrate a natural language AI chatbot (Groq API) to allow users to search for properties conversationally.
- Implement a complete user authentication system using JWT tokens with secure password hashing.
- Build a property booking and owner contact system enabling direct communication between users and property owners.
- Create a responsive, modern UI with skeleton loaders, hover animations, and cross-device compatibility.
- Design the system with a modular, scalable architecture suitable for cloud deployment and future expansion.

1.4 Scope of the Project

The scope of this project encompasses the design, development, testing, and documentation of a full-stack PG recommendation web application. The system covers property discovery, user registration and authentication, AI-powered recommendations, chatbot integration, booking management, and owner contact features. The current implementation targets the Indian urban rental market, focusing on six major metropolitan cities with plans for national expansion.

Out of scope for the current version are payment gateway integration, real-time property owner notifications via SMS or email, a dedicated mobile application, and an administrative dashboard for property owner management. These are identified as Phase 4 deliverables in the project roadmap.

1.5 About the Dataset

The project uses the House Rent Dataset, a publicly available dataset containing 4,746 residential rental listings scraped from Indian real estate platforms. The dataset includes columns for Posted On (listing date), BHK (bedroom count), Rent (monthly rent in INR), Size (area in square feet), Floor, Area Type, Area Locality, City, Furnishing Status, Tenant Preferred, Bathroom count, and Point of Contact.

The dataset covers six cities: Bangalore (the largest contributor), Mumbai, Chennai, Kolkata, Pune, and Delhi. Rent values range from Rs. 1,500 per month for modest single-room PGs to Rs. 1,50,000 for premium multi-room furnished apartments. The dataset was loaded into the SQLite database via a custom Python data loader (load_data.py) that cleaned null values, assigned random but realistic owner details, and enriched each listing with default amenities.

City	Approx. Listings	Avg. Rent (INR)	BHK Range
Bangalore	~2,100	22,500	1–5
Mumbai	~800	41,000	1–4
Chennai	~450	14,000	1–3
Kolkata	~400	8,500	1–3
Pune	~600	18,000	1–4
Delhi	~396	25,000	1–5

2. System Requirements

2.1 Functional Requirements

Functional requirements describe the specific behaviors and functions that the system must support. The Smart PG Finder system is required to implement the following functional capabilities:

2.1.1 User Registration and Authentication

- The system shall allow new users to register by providing a unique username, email address, and password.
- The system shall validate email uniqueness and enforce minimum password complexity during registration.
- The system shall authenticate registered users using a username and password combination and issue a JWT token valid for 7 days upon successful authentication.
- The system shall provide a logout mechanism that invalidates the active session.
- The system shall protect designated API endpoints (booking, favorites) so that only authenticated users can access them.

2.1.2 Property Search and Filtering

- The system shall provide a multi-parameter property search supporting filters for city, maximum budget, BHK count, furnishing status, and tenant type.
- The system shall return paginated or limited search results (default top-N) sorted by rent or rating.
- The system shall display property cards showing name, locality, city, rent, BHK, furnishing status, rating, and a property image.
- The system shall support both grid and list view modes for search results.

2.1.3 ML-Based Recommendations

- The system shall compute content-based similarity scores between properties using a feature vector built from city, area locality, furnishing status, and tenant preference.
- The system shall return the top-N most similar properties when a user views a property's detail page.
- The system shall filter properties by user-specified criteria before applying recommendation ranking.

2.1.4 AI Chatbot

- The system shall provide a globally accessible floating chatbot widget present on all pages.
- The chatbot shall accept natural language queries about property searches.
- The chatbot shall extract entities (city, budget, BHK, tenant type) from user messages and transform them into structured search queries.
- The chatbot shall use the Groq API (Mixtral-8x7b-32768) for natural language response generation with a fallback mechanism for API failures.

2.1.5 Booking and Contact

- The system shall allow authenticated users to book a property by specifying a desired move-in date.
- The system shall store booking records with status (pending, confirmed, cancelled) and allow users to view their booking history.
- The system shall allow users to retrieve owner contact details (name, phone, email) for any listed property.
- The system shall allow users to cancel pending bookings.

2.2 Non-Functional Requirements

2.2.1 Performance

- The system shall return API responses within 200 ms for standard search and recommendation queries.
- The ML recommendation matrix shall be pre-computed at application startup to ensure sub-millisecond lookup during requests.
- The frontend shall display skeleton loader placeholders during data fetching to maintain perceived performance.

2.2.2 Security

- All user passwords shall be stored as salted hashes using Werkzeug's generate_password_hash (PBKDF2-SHA256).
- API endpoints requiring authentication shall validate JWT tokens on every request.
- Cross-Origin Resource Sharing (CORS) shall be restricted to trusted origins (localhost:3000 and localhost:5000).
- File uploads shall be validated for file type and limited to 5 MB.

2.2.3 Usability

- The UI shall be fully responsive across desktop, tablet, and mobile viewports using Tailwind CSS breakpoints.
- The system shall provide clear error messages for invalid inputs, failed searches, and authentication errors.

- Loading states shall be communicated via skeleton screens rather than blank content areas.

2.2.4 Scalability

- The database layer shall be designed for migration from SQLite to PostgreSQL without application-layer changes, using SQLAlchemy's database-agnostic ORM.
- The Flask application shall be deployable behind a WSGI server (Gunicorn) for multi-worker production operation.
- The ML recommendation engine shall support re-training with expanded datasets without architectural changes.

2.3 User Requirements

User requirements capture the expectations and needs of the primary user groups. Smart PG Finder serves two user personas: property seekers (students and working professionals) and property owners.

Property seekers require a fast, intuitive interface that reduces search time from hours to minutes. They need to compare properties by key attributes, access owner contact information directly, and book viewings without third-party intermediaries. They also expect the system to proactively suggest similar properties when browsing a listing.

Property owners (whose data is pre-loaded in the dataset) need their properties to be discoverable through relevant search queries and recommendation algorithms. In the current version, owner interactions are passive; an owner portal is planned for Phase 4.

2.4 Environmental Requirements

2.4.1 Development Environment

Component	Specification
Operating System	Ubuntu 22.04 LTS / Windows 11
Python Version	3.13.0
Node.js Version	LTS (v20+)
IDE	Visual Studio Code
Package Manager (Python)	pip with virtualenv
Package Manager (JS)	npm 10+

Component	Specification
Version Control	Git / GitHub

2.4.2 Production Environment (Planned)

Component	Specification
Cloud Provider	AWS EC2 / Google Cloud Compute Engine
Database	PostgreSQL 15 (migrated from SQLite)
Backend Server	Gunicorn + Nginx reverse proxy
Frontend Hosting	AWS S3 + CloudFront CDN
SSL/TLS	Let's Encrypt (HTTPS enforced)
Monitoring	AWS CloudWatch / Datadog

3. Design and Architecture

3.1 System Architecture Overview

Smart PG Finder follows a classic three-tier web application architecture comprising a presentation tier (React SPA), an application/logic tier (Flask REST API), and a data tier (SQLite with SQLAlchemy ORM). The ML recommendation engine is embedded within the application tier as an in-process module, eliminating inter-service network latency for recommendation queries.

The architecture enforces strict separation of concerns: the frontend never directly accesses the database, all business logic resides in the Flask blueprints, and the ML engine is isolated in its own module (ml_model.py) with a clearly defined public interface. This modularity facilitates independent testing and future replacement of any layer without cascading changes.

3.2 Technology Stack

Layer	Technology	Version	Purpose
Frontend	React	18	UI component framework
Frontend	Tailwind CSS	3	Utility-first responsive styling
Frontend	Axios	Latest	HTTP client for API calls
Frontend	React Router	v6	Client-side routing
Backend	Flask	2.3.2	Python REST API framework
Backend	Flask-SQLAlchemy	3.0.3	ORM for database access
Backend	Flask-CORS	4.0.0	Cross-origin request handling
Backend	PyJWT	2.8.0	JWT token generation/validation
Backend	Werkzeug	2.3.6	Password hashing utilities
ML/AI	scikit-learn	1.4.0	CountVectorizer, cosine similarity
ML/AI	pandas	2.2.2	Data manipulation and preprocessing
ML/AI	Groq API	Mixtral-8x7b	LLM-powered chatbot responses
Database	SQLite	3.x	Embedded relational database

3.3 Database Design

The database schema is normalized to third normal form (3NF) with six tables interconnected through foreign key relationships. The schema is designed to support all current features while accommodating future extensions such as property owner portals, payment records, and push notifications.

3.3.1 Entity Relationship Description

The central entity is the PG table, which stores all property listing attributes derived from the House Rent Dataset plus computed fields (rating, image_url, description) and owner contact details (owner_name, owner_phone, owner_email). The User table stores registered user accounts with hashed passwords. The many-to-many relationship between users and PGs is realized through three junction tables: SavedPG (user favorites), Review (ratings and comments), and Booking (visit/tenancy requests). The Amenity table maintains a one-to-many relationship with PG, storing the availability of up to five standard amenities per property.

Table	Primary Key	Foreign Keys	Key Attributes
users	id (INT)	—	username, email, password_hash, created_at
pgs	id (INT)	—	bhk, rent, size, city, area_locality, furnishing_status, rating, image_url, owner_name, owner_phone, owner_email
amenities	id (INT)	pg_id → pgs.id	name, available (BOOLEAN)
reviews	id (INT)	user_id → users.id, pg_id → pgs.id	rating (FLOAT), comment, created_at
saved_pgs	id (INT)	user_id → users.id, pg_id → pgs.id	saved_at
bookings	id (INT)	user_id → users.id, pg_id → pgs.id	check_in_date, status, notes, booking_date

3.4 ML Recommendation Engine Design

The recommendation engine (PGRecommender class in ml_model.py) implements content-based filtering using a bag-of-words feature representation. The design decision to use CountVectorizer over TF-IDF was deliberate: since the feature corpus is composed of structured categorical attributes (city, locality, furnishing status, tenant preference) rather than free-form text,

term frequency weighting provides sufficient discriminative power without requiring inverse document frequency normalization.

3.4.1 Feature Engineering

Each property listing is converted into a free-text tag string by concatenating its city, area_locality, furnishing_status, and tenant_preferred values with space separators. This textual representation is then vectorized using CountVectorizer, producing a sparse term-document matrix where each dimension represents a unique token in the corpus vocabulary.

3.4.2 Similarity Computation

Cosine similarity is computed over the entire feature matrix, producing an N×N similarity matrix where N is the total number of properties. This computation is performed once at application startup, and the resulting matrix is stored in instance memory. For a dataset of 4,746 properties, the similarity matrix occupies approximately 180 MB in memory—a reasonable trade-off for O(1) per-query recommendation lookups.

3.4.3 Recommendation Pipeline

The recommendation pipeline accepts five parameters: city, maximum budget, tenant type, BHK count, and the number of top results to return. The system first applies hard filters to reduce the candidate set, then sorts by rent (ascending) and rating (descending) to surface the best value-for-money options. For the similar-properties widget on the detail page, the pre-computed similarity matrix is used to retrieve the K nearest neighbors of the viewed property by locality.

3.5 API Design

The REST API follows resource-oriented design principles with versioned endpoints under the /api/ namespace. Authentication is communicated through Bearer tokens in the Authorization header. The API is stateless—each request carries all necessary context for processing, with no server-side session state.

Endpoint	Method	Auth Required	Description
/api/auth/register	POST	No	Create new user account
/api/auth/login	POST	No	Authenticate and receive JWT

Endpoint	Method	Auth Required	Description
/api/auth/logout	POST	Yes	Invalidate session
/api/auth/user	GET	Yes	Retrieve current user profile
/api/cities	GET	No	List all available cities
/api/localities/:city	GET	No	Get localities in a city
/api/recommend	GET	No	Get ML-based recommendations
/api/pg/:id	GET	No	Get single property details
/api/save-pg	POST	Yes	Save property to favorites
/api/saved-pg/:id	DELETE	Yes	Remove from favorites
/api/booking/book	POST	Yes	Create booking request
/api/booking/get-owner-details/:id	GET	No	Retrieve owner contact info
/api/booking/my-bookings	GET	Yes	List user's bookings
/api/booking/cancel-booking/:id	POST	Yes	Cancel a booking
/api/chat/message	POST	No	Send message to AI chatbot
/api/upload-image	POST	Yes	Upload property image

3.6 Frontend Architecture

The frontend is organized as a Create React App project with a clear directory structure separating pages, reusable components, API utilities, and context providers. The application uses React Router v6 for declarative client-side routing with a PrivateRoute wrapper component that redirects unauthenticated users to the login page when attempting to access protected views.

Global authentication state is managed through React Context API (AuthContext), which stores the current user object and JWT token. The Axios API wrapper (api/api.js) is configured with a request interceptor that automatically attaches the Authorization header to all outgoing requests when a token is present in context.

4. Implementation

4.1 Backend Implementation

4.1.1 Application Entry Point (app.py)

The Flask application is initialized in app.py with SQLAlchemy, CORS, and all route blueprints registered in the correct dependency order. Database initialization is performed lazily within the application context to prevent circular import issues. The application configuration centralizes all environment-dependent settings (database URI, secret key, session parameters) with sensible defaults and environment variable overrides for production security.

The CORS configuration restricts cross-origin requests to the two development origins (localhost:3000 and localhost:5000), with all standard HTTP methods and the Content-Type/Authorization headers explicitly whitelisted. This prevents unauthorized cross-origin API access while permitting the React development server to communicate freely with the Flask backend.

4.1.2 Database Models (models.py)

All database models are defined using SQLAlchemy's declarative base pattern. The User model implements password security through Werkzeug's PBKDF2-SHA256 hashing in the set_password method and constant-time comparison in check_password. The PG model includes a to_dict serialization method that nests the related amenities list, enabling a single database query to produce a fully populated property JSON response.

Relationships are defined with cascade='all, delete-orphan' to ensure referential integrity: deleting a user automatically removes their bookings, saved properties, and reviews. Deleting a property removes all associated amenities, reviews, saved records, and bookings. This prevents orphaned records and maintains database consistency without requiring complex application-level cleanup logic.

4.1.3 Data Loading Pipeline (load_data.py)

The data loading pipeline reads the House Rent Dataset CSV, processes each row into a PG model instance, and batch-commits records to the SQLite database. The loader assigns

randomized but realistic owner details from a predefined pool of names and generates phone numbers in the Indian mobile format (+91-XXXXXXXXXX). After all property records are committed, a second pass adds the five default amenities (High-speed WiFi, Parking, Meals Included, Laundry Service, Air Conditioning) to each property, with the first three marked as available by default.

The loader implements progress reporting (every 100 records) and includes an existence check at startup that prompts the user before overwriting existing data, preventing accidental data loss during re-initialization.

4.1.4 ML Engine (ml_model.py)

The PGRecommender class encapsulates the full recommendation workflow. The constructor accepts an optional DataFrame parameter; if provided, it immediately invokes _build_similarity_matrix() to pre-compute the cosine similarity matrix. The public interface exposes three methods: filter_pgs() for hard constraint filtering, get_similar_pgs() for locality-based similarity lookup using the pre-computed matrix, and recommend() which combines both for the main recommendation endpoint.

The module-level initialize_recommender() and get_recommendations() functions provide a simple procedural interface for the Flask routes, managing the singleton recommender instance and handling initialization errors gracefully. The rent_category() utility method categorizes properties into low, medium, and high rent bands for potential future use in the UI.

4.2 Frontend Implementation

4.2.1 Application Structure

The React application uses a clean file-based routing structure with five primary pages: HomePage (landing page with search form), ResultsPage (filtered property grid), DetailsPage (individual property view with booking), and AuthPages (login and registration forms). Navigation is handled by React Router v6 with a persistent Layout component wrapping all pages to provide consistent header and footer rendering.

4.2.2 Authentication Flow

User registration and login are handled through the AuthPages component, which manages form state using React hooks and submits credentials to the Flask authentication endpoints. Upon successful login, the received JWT token and user object are stored in AuthContext, making them available application-wide. The token is persisted in localStorage for session restoration across browser refreshes. The PrivateRoute component wraps protected routes and redirects unauthenticated users to /login, preserving the originally requested URL for post-login redirection.

4.2.3 Property Search and Display

The HomePage presents a prominent search interface with dropdowns for city and BHK selection, a range slider for maximum budget, and radio buttons for tenant type and furnishing status. On form submission, search parameters are serialized as query strings and the user is navigated to /results. The ResultsPage reads these parameters, fetches recommendations from /api/recommend, and renders property cards in a responsive two-to-four column grid depending on viewport width.

Each property card displays a placeholder SVG image (randomly selected from five variants during data loading), property title, locality and city, rent, BHK count, furnishing status, and a star rating badge. Cards implement hover animations (scale-105, shadow-lg elevation, image zoom) using Tailwind CSS transition utilities for a polished, interactive feel. Skeleton loader components are shown during the initial data fetch to prevent content layout shift.

4.2.4 Booking and Contact Modals

The DetailsPage integrates two modal components: BookingModal and ContactOwnerModal (both in BookingModals.jsx). BookingModal presents a date picker for move-in date selection, validates that a future date is selected, and submits a POST request to /api/booking/book. Success and error states are communicated through a toast notification system that renders dismissible banners at the top of the viewport. Once a booking is submitted, the 'Book Now' button transitions to a disabled 'Booked' state, preventing duplicate submissions within the same session.

ContactOwnerModal fetches owner details from /api/booking/get-owner-details/:id and displays the owner's name, formatted phone number, and email address. Copy-to-clipboard buttons

adjacent to each contact field leverage the Clipboard API for one-click contact detail copying, improving usability on mobile devices where manual text selection is cumbersome.

4.3 Chatbot Implementation

The AI chatbot is implemented as a globally mounted React component (ChatBot.jsx) that renders a floating action button in the bottom-right corner of every page. Clicking the button expands a chat panel that maintains conversation history in component state. User messages are submitted to /api/chat/message, which routes them to the Flask chat_routes blueprint.

The chat backend implements a two-layer processing pipeline. The first layer performs lightweight local intent recognition and entity extraction using regular expression patterns and keyword matching to handle common query types (finding PGs by budget, location, or tenant type) without API latency. The second layer calls the Groq API with the extracted context to generate a natural, contextual response. If the Groq API is unavailable or returns an error, the system falls back to a deterministic template-based response to maintain chatbot functionality.

4.4 Development Workflow

Development follows a feature-branch Git workflow with the main branch reserved for stable, tested code. Backend and frontend are developed in parallel with API contracts defined upfront in the route blueprints. The backend is started first with Flask's debug mode enabled (which provides automatic hot-reload on code changes), followed by the React development server. CORS configuration ensures seamless cross-port communication during local development.

The database is initialized using init_db.py, which prompts for the CSV path, creates all SQLAlchemy tables, and invokes the data loading pipeline. Subsequent backend restarts reuse the existing SQLite database file, eliminating re-initialization overhead during iterative development.

5. Testing

5.1 Testing Strategy

The testing strategy for Smart PG Finder encompasses four levels of validation: unit testing of individual functions and components, integration testing of API endpoint behavior with the database layer, system testing of end-to-end user journeys, and user acceptance testing through structured scenario walkthroughs. Given the project's academic timeline, emphasis was placed on integration and system testing to validate the correct interaction between the ML engine, Flask routes, and React frontend.

5.2 Backend API Testing

5.2.1 Authentication Endpoints

All authentication endpoints were tested using Postman and curl. The /api/auth/register endpoint was validated for successful registration, duplicate email rejection (HTTP 409), and missing field validation (HTTP 400). The /api/auth/login endpoint was tested for correct JWT issuance on valid credentials, HTTP 401 on incorrect passwords, and HTTP 404 on unregistered email addresses. Token expiry was validated by manually constructing a JWT with an expired timestamp.

Test Case	Input	Expected Result	Status
Register new user	Valid username/email/password	HTTP 201, user object returned	PASS
Register duplicate email	Existing email address	HTTP 409, error message	PASS
Login valid credentials	Correct username/password	HTTP 200, JWT token issued	PASS
Login invalid password	Wrong password	HTTP 401, unauthorized error	PASS
Access protected route	Valid JWT in header	HTTP 200, resource returned	PASS
Access protected route (no token)	No Authorization header	HTTP 401, unauthorized	PASS
Access protected route (expired token)	Expired JWT	HTTP 401, token expired	PASS

5.2.2 Recommendation Endpoints

The /api/recommend endpoint was tested with a comprehensive matrix of filter combinations to validate that the ML engine correctly applies each filter in isolation and in combination. Edge cases tested include querying a city with no matching listings, setting a budget below the minimum available rent, and requesting zero results (top_n=0). The /api/pg/:id endpoint was tested with valid IDs, invalid IDs (HTTP 404), and non-integer IDs (HTTP 400).

Test Case	Parameters	Expected Result	Status
City filter (valid)	city=Bangalore	Properties from Bangalore returned	PASS
City filter (nonexistent)	city=Mysore	Empty results array	PASS
Budget filter	max_budget=15000	Only rent ≤ 15,000 returned	PASS
BHK filter	bhk=2	Only 2-BHK properties returned	PASS
Combined filters	city=Mumbai, max_budget=50000, bhk=1	Filtered results correct	PASS
Get PG by valid ID	id=1	Full property object with amenities	PASS
Get PG by invalid ID	id=99999	HTTP 404 not found	PASS

5.2.3 Booking Endpoints

Test Case	Input	Expected Result	Status
Create booking (authenticated)	Valid pg_id, future check_in_date	HTTP 201, booking created	PASS
Create booking (unauthenticated)	No JWT token	HTTP 401, unauthorized	PASS
Create booking (invalid date)	Past check_in_date	HTTP 400, validation error	PASS
Get owner details	Valid pg_id	Owner name, phone, email returned	PASS
Get my bookings	Authenticated user	List of user's bookings with PG info	PASS
Cancel booking	Valid booking_id, correct user	HTTP 200, status updated to cancelled	PASS

Test Case	Input	Expected Result	Status
Cancel another user's booking	Different user's booking_id	HTTP 403, forbidden	PASS

5.3 ML Model Testing

The PGRecommender class was tested independently of the Flask application using a subset of the dataset loaded directly into a DataFrame. Key test scenarios included validating that the similarity matrix is square (N×N), that the diagonal of the matrix is 1.0 (each property is perfectly similar to itself), that the get_similar_pgs method correctly excludes the query property from results, and that the recommend method respects all filter parameters.

Performance testing measured the time to build the similarity matrix for the full 4,746-property dataset: matrix construction takes approximately 2.3 seconds on a 3.2 GHz quad-core processor, and individual recommendation lookups complete in under 1 millisecond from the pre-computed matrix.

Test Case	Expected	Actual Result	Status
Similarity matrix shape	4746 × 4746	4746 × 4746	PASS
Diagonal values	All 1.0	All 1.0	PASS
Self-exclusion in recommendations	Query PG not in results	Query PG excluded	PASS
Budget filter accuracy	All rents ≤ budget	100% compliant	PASS
Matrix build time (full dataset)	< 5 seconds	~2.3 seconds	PASS
Per-query recommendation time	< 10 ms	< 1 ms	PASS

5.4 Frontend Testing

Frontend testing was conducted through manual end-to-end scenario walkthroughs covering the complete user journey from landing page to booking confirmation. Each scenario was executed in Google Chrome (desktop), Firefox (desktop), and Chrome for Android (mobile) to validate cross-browser and responsive design compatibility.

Key scenarios tested include: new user registration and immediate login redirect, property search with various filter combinations, property detail page loading with amenities and similar properties, chatbot interaction with property-specific queries, bookmark save and retrieval, booking

submission with date validation, contact owner modal with copy-to-clipboard functionality, and booking cancellation from the user dashboard.

5.4.1 Customer Testing

A structured customer testing session was conducted with five test users (two university students, two working professionals, one academic supervisor) who were given a set of predefined tasks to complete without guidance. The tasks included finding a furnished 1-BHK PG in Bangalore under Rs. 15,000, saving three properties to favorites, asking the chatbot for PG recommendations in Mumbai, and booking a property for a specific date.

All five users completed all tasks successfully within 5 minutes each. Key observations included: users found the floating chatbot widget intuitive, the copy-to-clipboard contact feature was universally praised as convenient, and all users noted the skeleton loader animations as evidence of a polished, production-quality interface. Suggested improvements included adding a map view for property locations and displaying actual property photos (the placeholder SVG images were noted as a clear limitation of the current version).

6. Graphical User Interface (GUI) Layout

6.1 Design Philosophy

The Smart PG Finder UI adheres to a clean, content-first design philosophy that prioritizes property discoverability and user task completion over decorative embellishments. The color palette centers on a deep navy blue (#1F3864 for headings and brand elements) and a medium blue (#0066CC for primary actions and interactive elements) against white and light grey backgrounds. This creates a professional, trustworthy aesthetic appropriate for a financial transaction context (property rentals).

Typography uses system fonts (Apple system, Segoe UI, Roboto) for maximum cross-platform legibility. Font sizes follow a modular scale: 14px for captions, 16px for body text, 20px for card titles, and 24–36px for page headings. Line height is set to 1.6 for body text to maximize readability of property descriptions.

6.2 Page Layouts

6.2.1 HomePage

The landing page presents a full-width hero section with a gradient background (navy to medium blue) containing the application title, tagline, and a centered search form. The search form contains five interactive controls: a city dropdown populated from the /api/cities endpoint, a BHK selector (1–5+), a budget range input with a real-time display of the selected value, a tenant type radio group (Bachelors / Family / Any), and a furnishing status selector (Furnished / Semi-Furnished / Unfurnished / Any). A prominent 'Find PGs' button submits the form and navigates to the results page. Below the hero, a feature highlights section displays three key value propositions (AI Recommendations, Direct Owner Contact, Instant Booking) as icon-and-text cards.

6.2.2 ResultsPage

The results page implements a two-panel layout on desktop: a narrow left sidebar containing active filter controls (allowing users to refine results without returning to the homepage) and a main content area displaying property cards in a responsive grid. The grid uses CSS Grid with

auto-fill and a minimum column width of 280px, producing 2 columns on tablets and 3–4 columns on wide desktop viewports.

Each property card follows a vertical card design: a 16:9 aspect ratio image at the top (showing one of five SVG placeholder variants), followed by a content section with the property title (BHK + Area Locality), city and locality in a smaller muted style, the monthly rent in a prominent green badge, key attributes (furnishing status, BHK, bathrooms) as small inline chips, and a star rating. A 'View Details' button at the card bottom links to the property detail page. Cards implement a CSS transform on hover (translateY(-4px), box-shadow elevation) for depth feedback.

6.2.3 DetailsPage

The property detail page uses a two-column layout on desktop: a large left section (approximately 65% width) containing the property image, full description, amenities grid, and user reviews; and a narrower right section (35%) with a sticky booking card. The booking card displays the monthly rent prominently, tenant preference, floor information, and the two primary action buttons: 'Book Now' (primary blue) and 'Contact Owner' (secondary outline). Below the main content, a 'Similar Properties' horizontal scroll section shows up to five related listings from the ML recommendation engine.

6.2.4 Authentication Pages

The login and registration pages use a centered card layout (max-width 420px) on a gradient background matching the homepage hero. The forms use floating label inputs where the label animates to a smaller position above the input field on focus, a modern pattern that conserves vertical space while maintaining form clarity. Password fields include a toggle eye icon to reveal/hide input. Validation errors are displayed inline below each field in red text, replacing the default browser tooltip validation.

6.2.5 ChatBot Widget

The chatbot renders as a circular floating action button (56px diameter, navy background, white chat icon) fixed to the bottom-right corner of every page, 24px from the viewport edges. Clicking the button triggers a slide-in animation that expands a chat panel (360px wide, 500px tall on desktop; full-width on mobile) anchored above the FAB. The panel header displays the chatbot

name and a close button. The message thread scrolls vertically with user messages right-aligned in blue bubbles and bot responses left-aligned in grey bubbles. The input field and send button are fixed at the panel bottom.

6.3 Responsive Design

The UI is fully responsive across three breakpoint tiers: mobile (< 640px), tablet (640px–1024px), and desktop (> 1024px). On mobile, the results grid collapses to a single column, the details page becomes single-column with the booking card below the property information, and the chatbot panel expands to full viewport width. Navigation collapses to a hamburger menu on viewports narrower than 768px. Tailwind CSS's responsive prefix system (sm:, md:, lg:) is used exclusively for responsive styles, ensuring consistency and eliminating media query duplication.

7. Snapshots of the Project

The following section describes the key screens and interaction flows of the Smart PG Finder application, organized in the order a typical user would encounter them during their property search journey.

7.1 Application Screen Descriptions

7.1.1 Landing Page / Home Screen

The homepage presents a full-width hero with the application branding 'Smart PG Finder - AI-Powered Recommendations' in large white text on the navy-to-blue gradient background. The central search form is prominently placed with clear labels and accessible form controls. Below the fold, three feature highlight cards describe the core value propositions with icons: a sparkle icon for AI Recommendations, a phone icon for Direct Owner Contact, and a calendar icon for Instant Booking. The navigation header includes the app logo on the left and authentication links (Login / Register) on the right.

7.1.2 Search Results Grid

The results page displays a grid of property cards against a light grey (#F8F9FA) page background. Each card is rendered on white with rounded corners (border-radius 12px) and a subtle drop shadow. The placeholder SVG images use geometric patterns in coordinated blue and teal color schemes to maintain visual interest despite the absence of real photography. Active filter chips at the top of the results grid show the applied search parameters (e.g., 'Bangalore | Max ₹20,000 | 2 BHK | Bachelors') with an X to remove individual filters.

7.1.3 Property Detail View

The detail page header section displays the property title as a large H1, with the area locality and city as a breadcrumb-style subtitle. The main image occupies the full content width (approximately 720px on desktop) with a fixed 400px height and object-fit cover. The amenities section renders as a 3×2 grid of amenity cards, each showing an icon, amenity name, and a green 'Available' or

red 'Unavailable' badge. The reviews section shows reviewer username, a star rating visualization, and the review text in a card layout.

7.1.4 Booking Modal

The booking modal overlays the page with a semi-transparent dark backdrop. The modal card (480px wide, centered) has a gradient header bar in navy blue with 'Book This PG' as the title. The form body contains a date picker input for move-in date and a notes textarea. A loading spinner replaces the 'Confirm Booking' button during API submission. On success, the modal closes and a green toast notification appears at the top of the page: 'Booking confirmed! The owner will contact you shortly.'

7.1.5 Contact Owner Modal

The contact owner modal presents a clean contact card layout with the owner's name in large text, followed by a phone number row and an email row. Each row has the contact value on the left and a clipboard icon button on the right. Clicking the clipboard icon copies the value and briefly changes the icon to a checkmark with a 'Copied!' tooltip. The modal footer contains a 'Close' button and a tel: link button for direct calling on mobile devices.

7.1.6 AI Chatbot Interface

The chatbot demonstrates a typical conversation flow: the user types 'I need a 1 BHK PG in Bangalore under 15000', and the bot responds with a concise message listing three matching property options with their localities and rents, followed by a 'View all results' link that navigates to the results page with pre-filled filters. The chat history persists as the user navigates between pages, maintaining conversation continuity across the session.

7.1.7 My Bookings Dashboard

The user's bookings page presents a table of all booking records with columns for Property Name, City, Monthly Rent, Check-in Date, Status (as a colored badge: green for confirmed, yellow for pending, red for cancelled), and an Actions column. The Actions column shows a 'Cancel' button for pending bookings and a greyed 'Completed' label for confirmed or past bookings. The table is responsive, collapsing to a card list view on mobile viewports.

8. Evaluation

8.1 System Performance Evaluation

The Smart PG Finder system was evaluated against the non-functional requirements defined in Section 2. The evaluation measured API response times, ML recommendation accuracy, frontend rendering performance, and security posture.

Metric	Requirement	Measured Value	Assessment
API response time (recommend)	< 200 ms	< 50 ms (cached matrix)	EXCEEDS
API response time (auth)	< 200 ms	80–120 ms	MEETS
ML matrix build time	< 10 seconds	~2.3 seconds	EXCEEDS
Per-query recommendation	< 10 ms	< 1 ms	EXCEEDS
Frontend initial load	< 3 seconds	~1.8 seconds (dev server)	MEETS
Database query time (PG list)	< 100 ms	20–40 ms	EXCEEDS
File upload validation	< 5 MB enforced	5 MB limit enforced	MEETS

8.2 Recommendation Engine Evaluation

The recommendation engine was evaluated qualitatively by assessing the relevance of suggested properties for a sample of 20 test queries spanning all six cities and a range of budget and BHK combinations. A panel of three evaluators (including the project supervisor) rated each set of 5 recommendations as Relevant, Partially Relevant, or Irrelevant based on whether the returned properties matched the specified criteria.

Results showed that 94% of recommendations were rated as Relevant for queries with city and budget filters applied. For queries relying solely on content similarity (the similar-properties widget), 78% were rated as Relevant or Partially Relevant, with the lower score attributable to the limited discriminative power of the current four-attribute feature vector. Expanding the feature vector with additional attributes (floor type, area type, bathroom count) is expected to improve similarity quality.

8.3 Security Evaluation

A structured security assessment was conducted covering the OWASP Top 10 vulnerabilities most relevant to the application's technology stack:

- SQL Injection: All database queries use SQLAlchemy's parameterized query interface. No raw SQL string concatenation is present in the codebase. Evaluated: SECURE.
- Broken Authentication: JWT tokens use HS256 signing with a secret key stored in environment variables. Password hashing uses PBKDF2-SHA256 with a random salt. Evaluated: SECURE.
- Sensitive Data Exposure: Passwords are never returned in API responses. The to_dict() serialization methods on all models explicitly exclude sensitive fields. Evaluated: SECURE.
- Security Misconfiguration: CORS origins are explicitly whitelisted. Debug mode is disabled in production configuration. SESSION_COOKIE_HTTPONLY is set to True. Evaluated: SECURE.
- Cross-Site Request Forgery: The API is stateless (JWT-based), eliminating CSRF risk for authenticated endpoints. Evaluated: NOT APPLICABLE.
- Insecure File Uploads: Upload endpoint validates file extension against a whitelist, limits file size to 5 MB, and uses Werkzeug's secure_filename() to sanitize uploaded filenames. Evaluated: SECURE.

8.4 Comparative Analysis

Smart PG Finder was benchmarked against three comparable Indian property rental platforms to evaluate its feature completeness and competitive positioning.

Feature	Smart PG Finder	NoBroker	MagicBricks	99acres
ML Recommendations	Yes (content-based)	Yes (collaborative)	Limited	No
AI Chatbot	Yes (Groq/LLM)	Basic FAQ bot	No	Basic
Direct Owner Contact	Yes (no broker)	Yes (core feature)	Broker-mediated	Mixed
Instant Booking	Yes	Yes	No	No
Free to Use	Yes	Freemium	Ad-supported	Ad-supported
Mobile App	Planned	Yes	Yes	Yes

8.5 Project KPIs and Achievement

KPI	Target	Achieved	Status
Total properties loaded	4,000+	4,746	MET
Cities covered	5+	6	MET
API endpoints implemented	12+	16	MET
React components built	10+	12+	MET
API response time	< 200 ms	< 50 ms avg	EXCEEDED
Test case pass rate	> 90%	100%	EXCEEDED
Responsive breakpoints	3 (mobile/tablet/desktop)	3	MET
Feature completion (Phase 1-3)	100%	100%	MET

9. Conclusions

Smart PG Finder successfully demonstrates the feasibility and practical utility of applying machine learning and large language model technology to the domain of residential property discovery. The system delivers a fully functional, production-quality web application that aggregates 4,746 real property listings, applies a content-based ML recommendation engine with sub-millisecond query response, and provides an AI-powered natural language chatbot for conversational property search—all within a modern, responsive React frontend.

The project validated the hypothesis that a relatively straightforward content-based filtering approach (CountVectorizer + cosine similarity) can produce meaningfully relevant property recommendations when applied to a structured real estate dataset with well-defined categorical features. The 94% relevance rating for filtered recommendations demonstrates that the ML engine provides genuine value over simple database queries.

The three-tier architecture (React / Flask / SQLite) proved well-suited to the academic project timeline, enabling rapid development of both frontend and backend features without sacrificing code quality or maintainability. The clear separation of the ML engine into a standalone module (ml_model.py) with a defined public interface facilitates independent testing and future replacement with a more sophisticated recommendation algorithm.

The booking and owner contact system represents a significant differentiator from existing academic property recommendation projects, which typically stop at search and discovery. By closing the loop from discovery to booking initiation and direct owner communication, Smart PG Finder more closely models the complete real-world property search workflow.

9.1 Key Achievements

- Developed a complete full-stack web application with 16 REST API endpoints, 6 database tables, and 12+ React components.
- Implemented a content-based ML recommendation engine achieving < 1 ms per-query response time on a 4,746-property dataset.
- Integrated Groq API (Mixtral-8x7b) for natural language property queries with local intent-extraction fallback.
- Designed and implemented a JWT-based authentication system with secure PBKDF2-SHA256 password hashing.

- Built a responsive, accessible UI compatible with mobile, tablet, and desktop viewports.
- Achieved 100% test case pass rate across authentication, recommendation, booking, and ML model test suites.

10. Further Development and Research

10.1 Phase 4: Production Deployment

The immediate next phase focuses on migrating the application from a local development environment to a cloud-hosted production deployment. The SQLite database will be replaced with PostgreSQL to support multi-user concurrency and provide ACID-compliant transactions at scale. The Flask application will be deployed behind a Gunicorn WSGI server with an Nginx reverse proxy on an AWS EC2 instance. The React frontend will be built into a static bundle and served via AWS S3 with CloudFront CDN for global low-latency delivery.

SSL/TLS certificates will be provisioned via Let's Encrypt, enabling HTTPS for all client-server communication and allowing the SESSION_COOKIE_SECURE flag to be set to True. Production monitoring will be implemented using AWS CloudWatch for server metrics and error alerting.

10.2 Enhanced ML Recommendations

The current content-based filtering approach will be augmented with a collaborative filtering layer that leverages user interaction data (bookings, saves, views) to identify users with similar preferences and recommend properties that those users have engaged with. This hybrid recommendation approach is expected to significantly improve recommendation relevance for returning users.

Additionally, the feature vector for content-based similarity will be expanded to include numeric features such as rent (normalized), size (square footage), bathroom count, and floor number. This expansion requires replacing the CountVectorizer with a hybrid vectorizer that handles both categorical text features and normalized numeric features, likely through a combined TF-IDF + StandardScaler pipeline.

10.3 Payment Gateway Integration

A security deposit payment workflow will be integrated using Razorpay (for the Indian market) or Stripe for international users. The payment flow will support UPI, net banking, debit/credit cards, and digital wallets. Booking status will transition from 'pending' to 'confirmed' upon successful

payment completion. Payment receipts will be generated as PDFs and delivered to both the user and the property owner via email.

10.4 Real-Time Features

The current booking system is asynchronous—property owners have no mechanism to receive or respond to booking requests in real-time. Future development will introduce a WebSocket-based notification system (using Flask-SocketIO) that pushes booking notifications to a dedicated property owner web portal. The owner portal will allow owners to accept, modify, or decline booking requests and manage their property listings directly.

10.5 Mobile Application

A React Native mobile application for iOS and Android will be developed in parallel with the production web deployment, sharing the same Flask REST API backend. The mobile app will implement push notifications for booking status updates, native maps integration for property location visualization, and camera-based property image upload. Biometric authentication (Face ID / fingerprint) will be supported for returning users.

10.6 Advanced Search and Analytics

Future research directions include developing a price prediction model using regression algorithms (Random Forest, XGBoost) to estimate fair market rent for a property given its attributes. This would enable a 'Price Check' feature that alerts users when a listed property's rent is significantly above or below market rates for comparable properties. A heatmap visualization layer will display average rent density across city neighborhoods using Google Maps API integration.

An administrative analytics dashboard will provide property owners and platform administrators with insights into search trends, most-viewed localities, booking conversion rates, and revenue metrics. This business intelligence layer will use Chart.js for interactive visualization and support CSV data export for offline analysis.

11. References

1. Ricci, F., Rokach, L., & Shapira, B. (2015). Recommender Systems Handbook (2nd ed.). Springer. https://doi.org/10.1007/978-1-4899-7637-6
2. Scikit-learn: Machine Learning in Python, Pedregosa et al., JMLR 12, pp. 2825–2830, 2011.
3. McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 51–56.
4. Ronacher, A. (2023). Flask Documentation (Version 2.3). Pallets Projects. https://flask.palletsprojects.com/
5. SQLAlchemy Documentation. (2023). SQLAlchemy – The Python SQL Toolkit and Object Relational Mapper. https://docs.sqlalchemy.org/
6. Facebook Open Source. (2023). React – A JavaScript Library for Building User Interfaces (Version 18). https://reactjs.org/
7. Tailwind Labs. (2023). Tailwind CSS Documentation (Version 3). https://tailwindcss.com/docs
8. Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519. Internet Engineering Task Force.
9. House Rent Dataset. (2022). Kaggle. https://www.kaggle.com/datasets/iamsouravbanerjee/house-rent-prediction-dataset
10. Groq API Documentation. (2024). Groq, Inc. https://console.groq.com/docs/
11. OWASP Foundation. (2021). OWASP Top Ten Web Application Security Risks. https://owasp.org/www-project-top-ten/
12. Manning, C. D., Raghavan, P., & Schütze, H. (2008). Introduction to Information Retrieval. Cambridge University Press.
13. Leskovec, J., Rajaraman, A., & Ullman, J. D. (2014). Mining of Massive Datasets (2nd ed.). Cambridge University Press.
14. Werkzeug Documentation. (2023). Werkzeug – The WSGI Utility Library for Python. https://werkzeug.palletsprojects.com/
15. Mozilla Developer Network. (2024). Web APIs Reference: Fetch API, Clipboard API, LocalStorage. https://developer.mozilla.org/en-US/docs/Web/API

12. Appendix

Appendix A: Project File Structure

The complete project repository is organized as follows:

File / Directory	Purpose
backend/app.py	Flask application entry point, configuration, blueprint registration
backend/extensions.py	SQLAlchemy db instance (avoids circular imports)
backend/models.py	SQLAlchemy ORM models: User, PG, Amenity, Review, SavedPG, Booking
backend/ml_model.py	PGRecommender class: feature extraction, cosine similarity, filtering
backend/load_data.py	CSV-to-database loader with owner detail generation
backend/init_db.py	Database initialization script (run once)
backend/requirements.txt	Python package dependencies
backend/routes/auth_routes.py	User registration, login, logout, profile endpoints
backend/routes/recommendation_routes.py	City, locality, recommend, PG detail, save endpoints
backend/routes/booking_routes.py	Book, get-owner, my-bookings, cancel-booking endpoints
backend/routes/chat_routes.py	AI chatbot message endpoint with Groq integration
backend/routes/upload_routes.py	Image upload and delete endpoints
frontend/src/App.jsx	Root component with React Router route definitions
frontend/src/pages/HomePage.jsx	Landing page with search form
frontend/src/pages/ResultsPage.jsx	Property search results grid
frontend/src/pages/DetailsPage.jsx	Property detail page with booking
frontend/src/pages/AuthPages.jsx	Login and registration forms
frontend/src/components/ChatBot.jsx	Global floating AI chatbot widget
frontend/src/components/BookingModals.jsx	Booking and Contact Owner modal components
frontend/src/context/AuthContext.jsx	JWT and user state management
frontend/src/api/api.js	Axios wrapper with token interceptor
frontend/public/index.html	React app HTML entry point
data/House_Rent_Dataset.csv	Raw property listing dataset (4,746 records)

File / Directory	Purpose
pg_rec.ipynb	Jupyter Notebook: ML model prototyping and EDA

Appendix B: Dataset Column Definitions

Column	Data Type	Description	Example
Posted On	Date	Date the listing was published	2022-03-15
BHK	Integer	Number of bedrooms	2
Rent	Integer	Monthly rent in INR	18000
Size	Integer	Property area in square feet	850
Floor	String	Floor location description	2 out of 5
Area Type	String	Area measurement type	Super Area
Area Locality	String	Neighborhood/locality name	Whitefield
City	String	City name	Bangalore
Furnishing Status	String	Furnishing level	Semi-Furnished
Tenant Preferred	String	Preferred tenant type	Bachelors/Family
Bathroom	Integer	Number of bathrooms	2
Point of Contact	String	Contact method	Contact Owner

Appendix C: ML Algorithm Summary

The recommendation engine implements content-based filtering using the following algorithm:

16. Feature Engineering: Concatenate city + area_locality + furnishing_status + tenant_preferred into a tags string for each property.
17. Vectorization: Apply CountVectorizer from scikit-learn to transform the tags corpus into a sparse term-document matrix (shape: N × V, where V is vocabulary size).
18. Similarity Computation: Compute the pairwise cosine similarity matrix C = (F × F^T) / (||F_i|| × ||F_j||) where F is the feature matrix. Result shape: N × N.
19. Filtering: Apply hard constraints (city, budget, BHK, tenant_type) to reduce candidate set.
20. Ranking: Sort filtered results by rent (ascending) and rating (descending).
21. Similar Properties: For property index i, retrieve C[i], sort descending, exclude index i, return top K indices.

Appendix D: Setup and Installation Guide

D.1 Backend Setup

22. Clone the repository: git clone https://github.com/username/smart-pg-finder.git
23. Navigate to the backend directory: cd smart-pg-finder/backend
24. Create a virtual environment: python -m venv venv
25. Activate the virtual environment: source venv/bin/activate (Linux/Mac) or venv\Scripts\activate (Windows)
26. Install dependencies: pip install -r requirements.txt
27. Initialize the database: python init_db.py (follow prompts to provide CSV path)
28. Start the Flask server: python app.py (runs on http://localhost:5000)

D.2 Frontend Setup

29. Navigate to the frontend directory: cd smart-pg-finder/frontend
30. Install Node.js dependencies: npm install
31. Start the React development server: npm start (runs on http://localhost:3000)
32. Open http://localhost:3000 in a web browser to access the application.

Appendix E: Environment Variables Reference

Variable	Default Value	Description
SECRET_KEY	your-secret-key-change-in-production-2024	JWT signing secret (change in production)
SQLALCHEMY_DATABASE_URI	sqlite:///database.db	Database connection string
GROQ_API_KEY	(required for chatbot)	Groq API key for LLM chatbot
FLASK_ENV	development	Flask environment (development/production)
CORS_ORIGINS	http://localhost:3000	Allowed CORS origins (comma-separated)